	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

Préparation : Création d'un nouveau projet avec Expo

1. Ouvrez Visual Studio Code et ouvrez un terminal.

2. Placez-vous dans le dossier dans lequel vous souhaitez créer votre projet.

3. Exécutez la commande suivante :

```
npx create-expo-app@latest
```

4. Lorsque l'assistant de création s'affiche, sélectionnez le template "blank".

5. Choisissez ensuite JavaScript comme langage du projet (et non TypeScript).

6. Entrez le nom de votre projet lorsque cela est demandé, puis attendez la fin de l'installation.

7. Accédez au dossier du projet :

```
cd nom-du-projet
```

8. Ouvrez ensuite le projet dans Visual Studio Code et vérifiez que les fichiers du projet Expo sont bien présents.

Remarque : pour ce TP, le projet doit obligatoirement être créé avec le template blank et JavaScript.

	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

Préparation : Lancer le projet Expo

1. Dans le terminal de Visual Studio Code, vérifiez que vous êtes dans le dossier du projet.

2. Lancez l'application avec :

```
npm start
```

3. Téléchargez l'application Expo Go depuis le Play Store sur votre téléphone.

4. Ouvrez Expo Go une fois l'installation terminée.

5. Scannez le code QR affiché dans le terminal de Visual Studio Code. Cela chargera votre projet Expo sur votre appareil mobile.

Une fois le projet correctement lancé, vous pouvez poursuivre avec les étapes existantes du TP4 : ouverture de App.js, ajout du composant Image, préparation des données, FlatList et création de l'écran produit.

	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

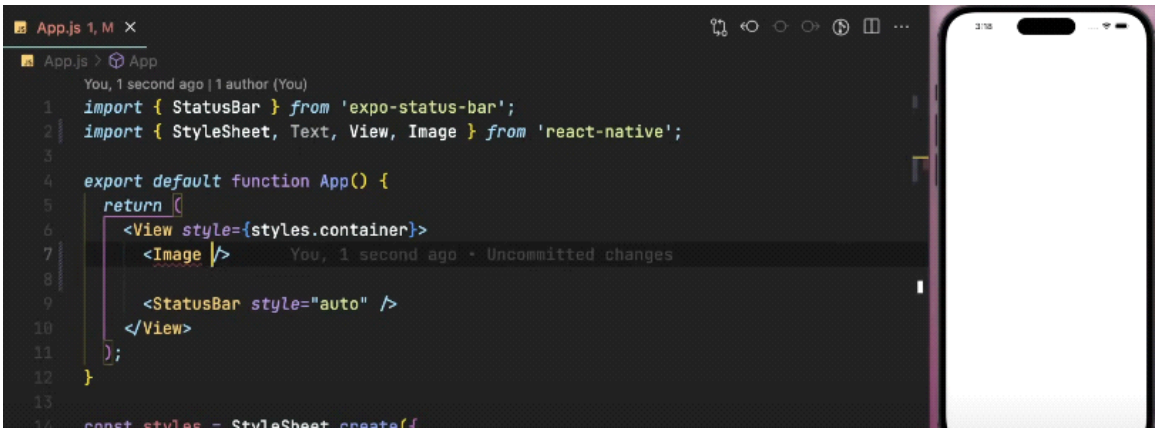
Étape 1 : Ouvrir le fichier de votre composant

Ouvrez le fichier principal de votre application, généralement App.js.

Étape 2 : Importer les composants nécessaires,Créer le composant d'image

Ajoutez les importations nécessaires en haut de votre fichier

Ajoutez le composant d'image à votre render ou votre fonction de retour dans le composant



ajouter au balise :

```
<Image
    source={{uri:'https://notjustdev-dummy.s3.us-east-2.amazonaws.com/nike/nike1.png' }}
    style={{width:'100%',aspectRatio:1,}} />
```

Dans le composant <Image> de React Native, les propriétés source et style jouent un rôle essentiel dans la manière dont l'image est affichée. on ecrit le style a l interieur de notre jsx le meilleur facon est de extraire nos style dans l objet styles

en bas afin que nous peux specifier que l image est egale aux styles que nous avon s defini et dans jsx on ecrit /





Étape 3 : Préparer les données

Créer un dossier src pour conserver tout le code source.

Créer un sous-dossier data dans src pour y mettre des données fictives products.js, cart.js

dans le products.js mettre cette code

> Remarque : le fichier data.js vous sera fourni par l'enseignante. Placez-le dans le dossier data prévu pour le TP.

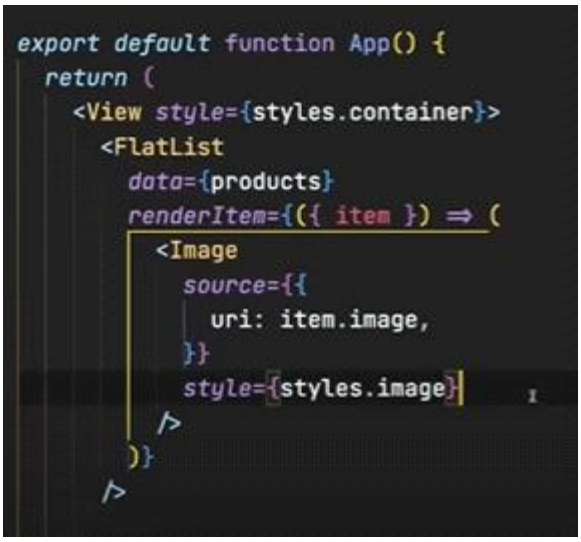
Étape 4 : Importer les données

// Dans app.js

import products from './src/data/products';

Étape 5 : Utiliser FlatList

// Dans app.js



<FlatList> :

Ce composant est utilisé pour afficher une liste déroulante d'éléments. Il est plus performant que ScrollView lorsqu'il y a une grande quantité d'éléments, car il ne charge que ceux qui sont visibles à l'écran.

data={products} : C'est la source de données. Ici, products est un tableau contenant des objets (par exemple des informations sur les produits). Chaque élément de la liste sera un produit.

renderItem={({ item }) => (...)} :

renderItem est une fonction qui définit la manière dont chaque élément de la liste sera affiché.

L'argument item représente un élément du tableau products. Il contient toutes les informations sur le produit (par exemple, nom, prix, image, etc.).

Étape 6 : Créer le composant d'affichage

<View style={styles.itemContainer}>

<Image

source={{ uri: item.image }} // Utilisation de l'URI de l'image

style={styles.image}

/>

```
        </View>
```

Dans l'objet style :

```
 itemContainer:{
  width:"50%",
  padding:1,
}
```

```
<FlatList  data={products} renderItem={({item})=>
        (<View style={styles.itemContainer}>
          <Image source={{uri:item.image}} style={styles.image}/>
          </View>)
        }
        numColumns={2} // Définit toujours 2 colonnes
        key={2} // Utilisez une clé statique pour éviter les changements dynamiques
        keyExtractor={(item) => item.id} // Assurez-vous que chaque élément a un id unique

        />
```

Étape 8 : Créer un écran produit

Créez un dossier screens dans src et un fichier ProductScreen.js.

Étape 9 : Intégrer l'écran produit dans l'application

```
// Dans app.js
import ProductScreen from './screens/ProductScreen';
const App = () => {
   return (
      <View>
         <ProductScreen />
      </View>
   );
};
```


3:35

	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

Étape 10 : Utiliser Git et GitHub

Git est un système de contrôle de version distribué qui permet aux développeurs de suivre les modifications apportées à des fichiers, de collaborer sur des projets et de gérer différentes versions d'un code source. Il facilite la collaboration en permettant à plusieurs utilisateurs de travailler sur le même projet simultanément tout en conservant un historique des modifications.

GitHub est une plateforme en ligne qui utilise Git pour héberger et gérer des dépôts de code source. Elle offre des fonctionnalités supplémentaires telles que la gestion des problèmes, la révision de code et des outils de collaboration pour faciliter le travail d'équipe. GitHub permet également de partager des projets avec d'autres développeurs et de contribuer à des projets open source.

Exécutez les commandes suivantes dans le terminal :

```
git status
git diff
git add .
git commit -m "change text to hello world"
git status
```

Explications :

- git status : affiche l'état actuel du dépôt pour voir les fichiers modifiés et non suivis.
- git diff : montre les différences entre les modifications non ajoutées et la dernière version des fichiers.
- git add . : ajoute tous les fichiers modifiés et nouveaux à l'index pour préparer le prochain commit.
- git commit -m : enregistre les modifications dans l'historique du dépôt. L'option -m permet d'ajouter directement le message du commit.

	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

Étape 11 : Connecter le projet à GitHub

Jusqu'à présent, les étapes du projet ont été réalisées localement sur votre ordinateur. Les modifications, configurations et tests sont effectués sur le système local. Vous pouvez ensuite partager le projet sur une plateforme comme GitHub.

1. Créez un nouveau dépôt sur GitHub sans l'initialiser avec un README.

2. Ajoutez l'URL du dépôt distant :

```
git remote add origin <URL-du-dépôt>
```

3. Renommez la branche principale en main :

```
git branch -M main
```

4. Envoyez le projet vers GitHub :

```
git push -u origin main
```

Après cette commande, votre projet local est lié au dépôt GitHub et la branche main est envoyée vers le dépôt distant.

	Institut Supérieur d'Informatique de Mahdia **(ISIMa)**
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée :** 2026/2027
TP : Dév.des.app.Mobiles	

Étape 12 : Effectuer une deuxième modification

1. Changez une nouvelle fois le fichier App.js en ajoutant du texte supplémentaire.

2. Ajoutez les nouvelles modifications à l'index :

```
git add .
```

3. Enregistrez cette deuxième modification dans l'historique :

```
git commit -m "2eme modification"
```

4. Envoyez la modification vers GitHub :

```
git push
```

Ces commandes permettent de conserver l'historique des changements et de synchroniser votre projet local avec le dépôt GitHub.

Résumé du cycle Git : modifier le code -> git add . -> git commit -m "message" -> git push.